

SHILATE

Software Hardware In Loop Automotive Testing Enviroment

Candidate: Tudor-Ciprian TĂMAȘ

Scientific coordinator: Sl. Dr. Ing. Valentin MĂRĂNESCU

Session: July 2026

ABSTRACT

The proliferation of software-defined vehicle (SDV) platforms has opened an opportunity to combine modern deep reinforcement learning (RL) with real automotive runtime stacks. This thesis presents **SHILATE** — a complete, open-source pipeline that trains a Proximal Policy Optimization (PPO) agent inside a Unity physics simulation and deploys the resulting policy onto an Eclipse Leda SDV stack, bridging the gap between laboratory RL research and realistic vehicle software deployment.

The simulation component is built in Unity 2022 LTS. A single rear-wheel-drive vehicle navigates a circular obstacle course instrumented with a 21-ray LiDAR-style sensor. The training bridge computes a shaped reward signal from progress, collision events, and episode milestones, and communicates with the Python training process exclusively over MQTT, keeping the two subsystems fully decoupled. A dedicated Editor window (*SHILATE Training Controller*) provides real-time health monitoring, metric graphs, and one-click training control without leaving the Unity Editor.

The Python training stack wraps the simulation in a Gymnasium-compliant environment, configures Stable-Baselines3 PPO with a two-layer actor-critic network, and streams structured telemetry back to the Editor. Trained checkpoints are subsequently deployed to the Eclipse Leda SDV image via a containerised MQTT-to-Kuksa bridge and a Velocitas vehicle application that enforces operational limits (overspeed, RPM redline) on the running agent.

Experimental results demonstrate that the PPO agent converges reliably on the obstacle course within 100 000 interaction steps and that the end-to-end deployment chain correctly propagates vehicle telemetry from the simulation through the VSS data layer to the Velocitas safety monitor. The project shows that an integrated sim-to-SDV pipeline is achievable with commodity open-source tooling, and identifies curriculum learning and multi-environment parallelisation as natural extensions for future work.

Keywords: reinforcement learning, software-defined vehicle, Unity simulation, PPO, MQTT, Eclipse Leda, Kuksa, Stable-Baselines3, Gymnasium.

CONTENTS

1	Introduction	4
1.1	Motivation and Context	4
1.2	Problem Statement	4
1.3	Objectives	5
1.4	Thesis Structure	5
2	Background and Related Work	6
2.1	Reinforcement Learning Fundamentals	6
2.2	Proximal Policy Optimization	6
2.3	Simulation-to-Real Transfer in Autonomous Driving	6
2.4	Software-Defined Vehicles and the VSS Data Model	7
2.5	MQTT and Publish-Subscribe Messaging	7
2.6	Related Work	7
3	System Architecture	9
3.1	High-Level Overview	9
3.2	Communication Backbone	9
3.3	Training Data-Flow	10
3.4	Configuration Management	10
3.5	Deployment Architecture on Eclipse Leda	11
4	Unity Simulation Design	12
4.1	Scene Construction	12
4.2	Vehicle Physics Model	12
4.3	Perception Layer	13
4.4	Reward Function	13
4.5	MQTT Bridge in Unity	14
4.6	Training Controller Editor Window	15
5	Python RL Training Pipeline	16
5.1	Gymnasium Environment Wrapper	16
5.2	PPO Agent Configuration	17
5.3	Training Loop and Health Telemetry	18
5.4	Inference Mode	19
5.5	Cross-Platform Process Management	20
6	SDV Integration with Eclipse Leda	21
6.1	Eclipse Leda Overview	21
6.2	MQTT-to-Kuksa Bridge	21
6.3	Velocitas Vehicle Application	22

6.4	Container Deployment	23
6.5	Leda Telemetry Broker	23
6.6	Raspberry Pi 4 Hardware Actuator	23
6.6.1	Circuit Design	24
6.6.2	Software: VSS Subscription to PWM Output	24
6.6.3	Connection to SHILATE	25
7	Experiments and Results	26
7.1	Experimental Setup	26
7.2	Training Convergence	26
7.3	Reward Shaping Ablation	27
7.4	Inference Evaluation	27
7.5	Health Monitoring Validation	27
7.6	End-to-End SDV Integration Test	28
8	Conclusions	29
8.1	Summary of Contributions	29
8.2	Limitations	29
8.3	Future Work	30
9	Bibliography	31

LIST OF FIGURES

LIST OF TABLES

3.1	MQTT topic map for the SHILATE training loop.	10
4.1	Key vehicle physics parameters.	12
5.1	Default PPO hyperparameters.	18
6.1	MQTT topic to VSS path mapping.	21
7.1	Experimental environment.	26
7.2	Training convergence milestones (mean over three seeds).	27
7.3	Inference evaluation over 50 episodes.	27

1. INTRODUCTION

1.1 MOTIVATION AND CONTEXT

The automotive industry is undergoing a fundamental architectural shift. Traditional vehicles are increasingly giving way to software-defined vehicles (SDVs) — platforms in which electronic control units are consolidated onto high-performance compute hardware and the vehicle's behaviour is governed by software layers that can be updated over the air [1]. This transformation enables capabilities that were previously inconceivable: dynamic reconfiguration of driving characteristics, continuous safety-critical updates without a workshop visit, and the integration of machine-learning-based components directly into the vehicle runtime.

At the same time, deep reinforcement learning (RL) has matured into a practical engineering tool. Algorithms such as Proximal Policy Optimization (PPO) [2] achieve human-competitive or superhuman performance in continuous control tasks, with comparatively modest tuning effort. The natural question that arises is: *how can an RL-trained policy be moved, end-to-end, from a training simulation onto an SDV runtime stack in a reproducible and maintainable way?*

This question is not merely academic. Automotive-grade deployment requires that a trained model run inside a standardised software container, interact with vehicle signals through a well-defined data layer, and remain subject to functional-safety monitors that can intervene when the model operates outside its intended envelope. Bridging this gap between laboratory RL and production SDV middleware is the core motivation behind the work presented in this thesis.

1.2 PROBLEM STATEMENT

Existing RL research in the autonomous driving domain concentrates almost exclusively on training performance — reward curves, sample efficiency, and policy robustness [3]. The software engineering challenge of taking a trained checkpoint and deploying it as a first-class citizen of an SDV runtime stack is largely unexplored in the open-source community. Concretely, the following sub-problems remain unaddressed in an integrated, open-source pipeline:

1. **Simulation-to-middleware coupling.** A physics simulator and an SDV middleware stack speak fundamentally different protocols; a communication bridge that is both low-latency and decoupled from either system is needed.
2. **Standardised vehicle-signal representation.** Raw simulator outputs must be mapped onto industry-standard Vehicle Signal Specification (VSS) paths so that the SDV safety layer can interpret them without simulator-specific knowledge.
3. **Operational safety monitoring.** A deployed RL policy may extrapolate outside its training distribution; a lightweight, declarative safety monitor must be able to flag

or override the policy's actions in real time.

4. **Reproducible training workflow.** The entire pipeline — from launching the simulation to saving a deployable checkpoint — must be reproducible from a single control surface without manual orchestration.

1.3 OBJECTIVES

This thesis addresses the above problems through the design, implementation, and evaluation of **SHILATE** (Software-defined-vehicle Hardware-In-the-Loop Autonomous Training Environment). The specific objectives are:

1. Design and implement a Unity-based physics simulation of a rear-wheel-drive vehicle navigating an instrumented obstacle course.
2. Define a shaped reward function that guides a PPO agent to complete laps reliably and safely, and expose a Gymnasium-compliant Python interface over MQTT.
3. Implement a real-time health-monitoring and training-control layer inside the Unity Editor so that a researcher can observe and intervene in a training run without leaving the development environment.
4. Construct an SDV deployment chain using Eclipse Leda, the Kuksa vehicle signal databroker, and a Velocitas vehicle application that enforces operational limits on the running policy.
5. Evaluate the pipeline quantitatively: training convergence, inference lap performance, and end-to-end signal propagation through the SDV stack.

1.4 THESIS STRUCTURE

The remainder of this document is organised as follows. Chapter 2 reviews theoretical background in reinforcement learning, SDV middleware, and communication protocols, and situates SHILATE among related work. Chapter 3 presents the overall system architecture. Chapter 4 describes the Unity simulation in detail. Chapter 5 documents the Python RL training pipeline. Chapter 6 explains the SDV integration through Eclipse Leda. Chapter 7 reports experimental results. Chapter 8 concludes with a summary of contributions and future work.

2. BACKGROUND AND RELATED WORK

2.1 REINFORCEMENT LEARNING FUNDAMENTALS

Reinforcement learning is a computational framework for sequential decision making under uncertainty [4]. An agent interacts with an environment modelled as a Markov Decision Process (MDP), defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$ is the transition kernel, $R : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the reward function, and $\gamma \in [0, 1)$ is the discount factor. The agent's goal is to learn a policy $\pi : \mathcal{S} \rightarrow \mathcal{A}$ that maximises the expected discounted return $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$.

Policy gradient methods optimise the policy directly via the gradient of the expected return with respect to policy parameters θ . The fundamental result is the policy gradient theorem:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a|s) \cdot Q^{\pi_{\theta}}(s, a)]. \quad (2.1)$$

Actor-critic architectures maintain both a policy network (actor) and a value network (critic) trained simultaneously; a learned baseline $V_{\phi}(s)$ reduces gradient variance through the advantage estimate $A(s, a) = Q(s, a) - V_{\phi}(s)$.

2.2 PROXIMAL POLICY OPTIMIZATION

Proximal Policy Optimization [2] takes the largest possible improvement step on a policy without destabilising training. It replaces Trust Region Policy Optimization's constrained optimisation with a clipped surrogate objective:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon) \hat{A}_t \right) \right], \quad (2.2)$$

where $r_t(\theta) = \pi_{\theta}(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ is the probability ratio, $\hat{A}_t$ is the estimated advantage, and ε (typically 0.2) is the clipping threshold. The full objective adds a value-function regression term and an entropy bonus:

$$L(\theta) = L^{\text{CLIP}}(\theta) - c_1 L^{\text{VF}}(\theta) + c_2 H[\pi_{\theta}(\cdot|s_t)]. \quad (2.3)$$

PPO is well suited to the vehicle control domain because its continuous Gaussian policy maps naturally onto a steer/throttle/brake action space, and its on-policy rollout structure integrates cleanly with a wall-clock-rate simulation running in a separate process. The Stable-Baselines3 library [5] provides the production implementation used in this project, validated against reference benchmarks.

2.3 SIMULATION-TO-REAL TRANSFER IN AUTONOMOUS DRIVING

Training autonomous driving policies in simulation rather than on physical hardware is standard practice, motivated by safety, cost, and the ability to reset episodes deterministically [3]. The primary risk is the *sim-to-real gap*: discrepancies between simulated and real sensor

readings, physics, and environmental conditions cause policies that perform well in simulation to degrade on deployment.

Prominent platforms include CARLA [6], an Unreal-Engine-based open-source simulator modelling urban driving at high visual fidelity, and the Unity ML-Agents Toolkit [7], which provides a general-purpose RL training framework tightly integrated with the Unity Editor. SHILATE uses raw Unity physics without ML-Agents for two reasons: it avoids an opaque intermediate API layer, and the training process communicates exclusively through MQTT — the same protocol used by the target SDV stack — so no additional bridge is required for deployment.

2.4 SOFTWARE-DEFINED VEHICLES AND THE VSS DATA MODEL

The COVESA Vehicle Signal Specification (VSS) [8] is a hierarchical, vendor-neutral data model for vehicle signals. VSS paths follow a dotted-name convention, e.g. `Vehicle.Speed` (km/h), `Vehicle.Powertrain.CombustionEngine.Speed` (RPM), and `Vehicle.Chassis.SteeringWheel.Angle` (degrees).

The Eclipse Kuksa databroker [9] implements a VSS-aligned gRPC server acting as the vehicle signal hub. Producers write to VSS paths; consumers subscribe to changes. Eclipse Velocitas [10] builds on Kuksa with a vehicle-application SDK and deployment scaffold. In SHILATE, the Velocitas application monitors the running RL agent for speed and RPM limit violations.

2.5 MQTT AND PUBLISH-SUBSCRIBE MESSAGING

MQTT 3.1.1 [11] is a lightweight publish-subscribe protocol designed for constrained IoT environments. A central broker decouples producers and consumers: a publisher pushes a message to a topic string and all subscribers to that topic receive it with no direct network coupling between the two parties. SHILATE uses QoS 0 (at-most-once delivery) throughout, which minimises latency at the cost of possible message loss — acceptable in a training loop where the next observation replaces the current one within tens of milliseconds. The MQTT protocol is also natively supported by the Eclipse Leda SDV image, which ships Mosquitto as a core service.

2.6 RELATED WORK

A substantial body of work addresses RL for autonomous driving within simulation [3]. OpenAI's `gym-car-racing` [12] provides a top-down racetrack benchmark but offers no path to real vehicle deployment. CARLA-based RL studies [6] achieve high visual realism but operate as closed systems with no integration to SDV middleware. The Unity ML-Agents toolkit [7] enables RL training within Unity but couples the Python trainer tightly to Unity's side-channel API, making it difficult to reuse the same environment against a physical vehicle.

From the SDV side, the Eclipse Kuksa and Velocitas communities have demonstrated containerised vehicle applications that read VSS signals and enforce policies, but without an RL training pipeline attached. SHILATE contributes the missing link: a training-to-deployment

chain that is agnostic to both the simulation engine and the SDV stack as long as both speak MQTT and VSS.

3. SYSTEM ARCHITECTURE

3.1 HIGH-LEVEL OVERVIEW

SHILATE is structured as three loosely coupled tiers, each running as an independent process or container and communicating exclusively through a Mosquitto MQTT broker. This design ensures that any tier can be replaced or upgraded without modifying the others, and that the same communication fabric used during training is available unchanged on the deployment target.

Tier 1 — Unity Simulation. A Unity 2022 LTS Editor instance runs the vehicle physics, the obstacle course, and the sensor model. It publishes observations and reward signals to MQTT and consumes control commands from the same broker. The Unity process also hosts the *Training Controller* Editor window, which provides a developer-facing UI for health monitoring and training control.

Tier 2 — Python RL Trainer. A Python process wraps the MQTT interface in a Gymnasium-compliant environment [13], instantiates a Stable-Baselines3 PPO agent [5], and runs the training loop. It emits structured telemetry back through MQTT so the Editor window can display live metrics. After training, the same Python process can be re-invoked in inference mode with a saved checkpoint.

Tier 3 — Eclipse Leda SDV Stack. The Eclipse Leda SDV image [1] runs on QEMU or physical hardware. It hosts a Mosquitto broker, a Kuksa databroker, an `mqtt-kuksa-feeder` container that maps MQTT telemetry to VSS paths, and a Velocitas vehicle application that monitors operational limits.

3.2 COMMUNICATION BACKBONE

All inter-tier communication passes through a single Mosquitto MQTT 3.1.1 broker [11]. The topic namespace uses the fixed prefix `env0/`, reflecting the single-environment design. The complete topic set is listed in Table 3.1.

Table 3.1: MQTT topic map for the SHILATE training loop.

Topic	Direction	Payload
env0/leda/control/steer	Python→Unity	float $[-1, 1]$
env0/leda/control/throttle	Python→Unity	float $[0, 1]$
env0/leda/control/brake	Python→Unity	float $[0, 1]$
env0/leda/control/reset	Python→Unity	"1"
env0/vehicle/speed	Unity→Python	float km/h
env0/vehicle/training/obs	Unity→Python	JSON array
env0/vehicle/training/reward	Unity→Python	float
env0/vehicle/training/done	Unity→Python	bool
env0/vehicle/training/episode_end	Unity→Editor	JSON object
env0/vehicle/training/heartbeat	Unity→Editor	JSON (1 Hz)

QoS 0 (at-most-once delivery) is used throughout. Given that observations arrive at 10 Hz and each new observation makes the previous one irrelevant, occasional message loss is harmless and the absence of acknowledgement handshakes reduces loop latency significantly compared to QoS 1 or 2.

3.3 TRAINING DATA-FLOW

Each training step follows a synchronised request-response pattern over MQTT. The Python environment calls `step(action)`, which:

1. Publishes steer, throttle, and brake to the three control topics.
2. Blocks on a threading event until Unity publishes a new `training/obs` message, or a 5-second timeout elapses.
3. Reads the co-published reward and done values.
4. Returns `(obs, reward, terminated, truncated, info)` to Stable-Baselines3.

This cycle runs at the Unity physics rate (approximately 50 Hz wall-clock), throttled by the 10 Hz sensor publication interval. The PPO rollout buffer accumulates 2048 transitions before a gradient update.

3.4 CONFIGURATION MANAGEMENT

Configuration is split across three locations:

- **config.json** (repository root): holds `model.ray_count` (default 9 in the root copy; the simulation runs 21 rays). Read by `config_loader.py` with fallback.
- **TrainingSettings.asset** (Unity ScriptableObject): persists training knobs configured through the Editor window — MQTT host/port, virtual-environment path, script paths, PPO hyperparameters, checkpoint save location, and optional resume-model path. Survives Editor restarts without an on-disk config file.

- **CLI flags** (`train.py`): hyperparameters from `TrainingSettings` are forwarded as command-line arguments when `PythonProcessManager` spawns the trainer, keeping the Python process stateless with respect to the Editor.

3.5 DEPLOYMENT ARCHITECTURE ON ECLIPSE LEDA

When a trained checkpoint is deployed onto the Leda image, three containers run simultaneously:

1. **leda-controller** (inference mode): loaded with the `best_model.zip` checkpoint; subscribes to `env0/vehicle/training/obs` and publishes control commands.
2. **mqtt-kuksa-feeder**: bridges `env0/vehicle/#` telemetry topics to VSS paths in the Kuksa databroker via gRPC.
3. **velocitas-app**: subscribes to `Vehicle.Speed` and engine RPM in the Kuksa databroker and publishes alerts to `leda/command/alert` when thresholds are exceeded.

Each container is described by a Kanto manifest (`kanto-manifest.json`) specifying the OCI image reference, environment variables, and restart policy. The provisioning script `leda/provision-device.sh` copies manifests to the device and triggers Kanto to pull and start the containers.

4. UNITY SIMULATION DESIGN

The simulation is not just a backdrop – it is the only reality the agent ever experiences during training. Every aspect of the environment, from how the car steers around a corner to what penalty it receives for scraping a wall, directly shapes the policy that eventually runs on the real vehicle. This chapter explains the design choices made for the Unity simulation and why they were made that way.

4.1 SCENE CONSTRUCTION

The simulation scene is constructed programmatically at runtime rather than authored in the Unity Editor. `RuntimeSceneBuilder.cs` runs on scene load and checks whether the required game objects are present; if absent, it invokes `CarFactory` to spawn the full vehicle hierarchy and `SpawnObstacleCourse` to instantiate the track. This approach guarantees a well-defined initial state regardless of how the Editor project was last saved.

The obstacle course is a circular ring generated by `ObstacleCourse.cs`. The ring is bounded by two concentric cylindrical walls: inner radius 25 m, outer radius 40 m, giving a track width of 15 m. Twelve obstacle objects are distributed at equal angular intervals. Wall height is 3 m, sufficient to be detected by raycasts but low enough to avoid occluding the follow camera. The vehicle is spawned at position (32.5, 0.5, 0) — on the track centreline facing the initial direction of travel.

4.2 VEHICLE PHYSICS MODEL

The vehicle is a rear-wheel-drive configuration assembled by `CarFactory.cs`. The chassis mass is 1500 kg. Four `WheelCollider` components provide the physics contact model using Unity's [14] built-in Pacejka-inspired tyre model. Key parameters are listed in Table 4.1.

Table 4.1: Key vehicle physics parameters.

Parameter	Value	Unit
Chassis mass	1500	kg
Maximum motor torque	450	Nm
Maximum steer angle	35	degrees
Final drive ratio	9.0	—
Maximum brake torque	3000	Nm
Regenerative brake torque	150	Nm
Maximum speed	150	km/h

`VehicleController.cs` translates the three-component action vector $[a_{\text{steer}}, a_{\text{throttle}}, a_{\text{brake}}]$ received from `RemoteDriveInput.cs` into `WheelCollider` motor torques and steer angles. A creep behaviour applies a small constant torque in drive gear

when both throttle and brake inputs are near zero, mirroring a real automatic-transmission vehicle.

4.3 PERCEPTION LAYER

The vehicle's sole sensor is a LiDAR-inspired raycast fan implemented in `RaycastSensor.cs`. Twenty-one rays are cast in the vehicle's horizontal plane, spanning a 120° arc centred on the forward axis, with a range of 50 m. The raycast loop itself is compact:

Listing 4.1: Ray fan loop from `RaycastSensor.cs`.

```

1 void CastRays()
2 {
3     Vector3 origin = transform.TransformPoint(rayOriginOffset);
4     float halfSpread = raySpreadAngle * 0.5f;
5     float step = rayCount > 1 ? raySpreadAngle / (rayCount - 1) : 0f;
6
7     for (int i = 0; i < rayCount; i++)
8     {
9         float angle = -halfSpread + step * i;
10        Vector3 direction = Quaternion.Euler(0f, angle, 0f) * transform.
            forward;
11
12        if (Physics.Raycast(origin, direction, out RaycastHit hit,
            rayLength, hitLayers))
13            _distances[i] = hit.distance / rayLength; // 0 = close, 1 =
                max range
14        else
15            _distances[i] = 1f; // no obstacle
                detected
16    }
17 }
```

Dividing by `rayLength` normalises each reading into $[0, 1]$: a value of 1 means the ray reached full range without hitting anything while values near 0 indicate an obstacle very close to the bumper. Each ray returns a normalised distance $d_i \in [0, 1]$, and the sensor publishes the full array to `env0/vehicle/training/obs` at 10 Hz.

The complete observation vector received by Python is:

$$\mathbf{o} = \left[d_0, \dots, d_{N-1}, \frac{v}{v_{\max}}, \frac{\delta + 1}{2} \right] \in \mathbb{R}^{N+2}, \quad (4.1)$$

where N is the configured ray count, v is current speed (km/h), $v_{\max} = 150$ km/h, and δ is steer angle normalised to $[-1, 1]$. All elements of $\mathbf{o}$ lie in $[0, 1]$, simplifying network initialisation.

4.4 REWARD FUNCTION

The per-step reward is:

$$r_t = r_{\text{progress}} + r_{\text{collision}} + r_{\text{milestone}}, \quad (4.2)$$

where:

- $r_{\text{progress}} = 2 \cdot \Delta p_t$ — signed change in track angle progress (degrees) since the last step.
- $r_{\text{collision}} = -5$ on any step where the vehicle reports a collision contact (applied once per event).
- $r_{\text{milestone}} = +50$ on full lap completion; $+25$ on first half-turn of the course (at most once per episode each).

The reward shaping is implemented in the `FixedUpdate()` loop of `TrainingBridge.cs`. Collision detection uses `OnCollisionEnter` rather than the raycast array, preventing any edge-case where a collision could be missed if the sensor update and physics update happened to occur in the wrong order:

Listing 4.2: Reward computation excerpt from `TrainingBridge.cs`.

```

1 float angleDelta = Mathf.DeltaAngle(_lastAngle, currentAngle);
2 _lastAngle = currentAngle;
3 _totalAngleProgress += angleDelta;
4
5 float stepReward = angleDelta * progressReward; // progressReward = 2
6
7 bool collided = _collidedThisStep;
8 _collidedThisStep = false;
9 if (collided)
10     stepReward += collisionPenalty; // collisionPenalty =
11     -5
12
13 if (!_halfTurnAwarded)
14 {
15     float heading = vehicle.transform.eulerAngles.y;
16     if (heading >= 170f && heading <= 190f)
17     {
18         stepReward += halfTurnBonus; // halfTurnBonus = 25
19         _halfTurnAwarded = true;
20     }
21 }
22
23 if (obstacleCourse != null && _totalAngleProgress >= 360f)
24 {
25     stepReward += finishBonus; // finishBonus = 50
26     finished = true;
27 }

```

An episode terminates on lap completion, collision, or after the 60-second wall-clock timeout. Timeout terminations are reported as `truncated=True` so that Stable-Baselines3 bootstraps the value estimate rather than treating the transition as a terminal failure.

4.5 MQTT BRIDGE IN UNITY

Unity does not ship a production-grade MQTT client. SHILATE includes a minimal custom `MqttClient.cs` that implements the MQTT 3.1.1 wire protocol [11] natively in C# without external dependencies. The client handles `CONNECT`, `PUBLISH`, `SUBSCRIBE`, `PINGREQ/PINGRESP`, and `DISCONNECT` packet types with a 30-second keep-alive.

`LedaBroker.cs` provides the application-facing API, configured via `LedaBroker.Configure(host, port, "env0")` at startup. It publishes vehicle telemetry at 10 Hz and receives control commands. A Python heartbeat timeout (5-second threshold) detects when the trainer has stopped sending commands and holds the vehicle stationary until reconnection. This is a small but important detail — without it, a crashed Python process would leave the simulated car coasting forward indefinitely into a wall.

4.6 TRAINING CONTROLLER EDITOR WINDOW

The *SHILATE Training Controller* (`SHILATE` → Training Controller) is an `EditorWindow` that eliminates manual terminal management during training. Its layout from top to bottom comprises:

1. **Toolbar**: training duration and MQTT connection indicator.
2. **Health banner**: colour-coded strip showing the current state (*Idle*, *Healthy*, *Warning*, *Critical*, *Disconnected*) with explicit problem text.
3. **Snapshot row**: episode count, last-episode reward, 10-episode rolling mean, and heartbeat counter.
4. **Settings panel** (collapsible): all `TrainingSettings` fields exposed as GUI controls.
5. **Metric graphs**: scrolling plots of episode reward mean, policy gradient loss, value loss, and KL divergence.
6. **Issue log**: timestamped list of every health-state transition for the session.
7. **Controls**: *Start Training* / *Run Model* / *Stop*.

The *Start Training* button enters Play mode programmatically and, once the scene is loaded and the MQTT connection is confirmed, launches the Python training script with arguments from `TrainingSettings`. The health evaluator monitors five independent failure modes: MQTT disconnection, Python process crash, observation gap exceeding 5 seconds, heartbeat/reward gap exceeding 60 seconds, and reward collapse (more than 50% drop over a 10-episode window).

5. PYTHON RL TRAINING PIPELINE

Getting a reinforcement learning agent to drive a virtual car is, in principle, straightforward. The tricky part is wiring everything together so the agent can actually talk to the simulation, watch what is happening in real time, and resume a training run without losing work. This chapter walks through how SHILATE does all three.

5.1 GYMNASIUM ENVIRONMENT WRAPPER

The Python side of SHILATE speaks Gymnasium [13] — the standard interface that practically every modern RL library expects. The wrapper class `ShilateEnv` lives in `environment.py` and translates between the clean Gym API and the somewhat messier world of MQTT messages flying back and forth with Unity.

The observation and action spaces are declared once in `__init__` and never change at runtime:

Listing 5.1: Observation and action space declarations from `environment.py`.

```
1 obs_size = ray_count + 2          # rays + normalised speed + normalised
   steer
2 self.observation_space = spaces.Box(
3     low=0.0, high=1.0, shape=(obs_size,), dtype=np.float32
4 )
5 self.action_space = spaces.Box(
6     low=np.array([-1.0, 0.0, 0.0], dtype=np.float32),
7     high=np.array([ 1.0, 1.0, 1.0], dtype=np.float32),
8     dtype=np.float32,              # [steer, throttle, brake]
9 )
```

Keeping everything in $[0, 1]$ means the first layer of the network always receives well-scaled inputs – no normalisation wrapper needed.

The `step()` method is where the action-observation loop actually happens. It sends the action to Unity over MQTT, then simply waits:

Listing 5.2: The core step loop from `environment.py`.

```
1 def step(self, action):
2     steer, throttle, brake = float(action[0]), float(action[1]), float(
3         action[2])
4     self._ctrl.send_action(steer, throttle, brake)
5
6     self._ctrl.obs_event.clear()
7     if not self._ctrl.obs_event.wait(timeout=OBS_TIMEOUT):
8         # Unity stopped responding -- truncate rather than crash
9         _emit("SHILATE-HEALTH",
10             f"obs_timeout phase={step} step={self._step_count}")
```

```
10         return self._build_observation(), -1.0, False, True, {"reason":  
11             "obs_timeout"}  
12     obs      = self._build_observation()  
13     reward   = self._ctrl.training_reward  
14     done     = self._ctrl.training_done  
15     self._step_count += 1  
16     return obs, reward, done, False, {}
```

`threading.Event.wait()` releases the Python GIL while blocking, so the MQTT callback thread can still receive messages. If Unity goes silent for more than five seconds — say because the Editor was paused or crashed — the episode is marked `truncated=True` and Stable-Baselines3 bootstraps the value from the last state instead of applying a terminal penalty.

The MQTT connection lives in `controller.py`. The `VehicleController` class maintains a `paho-mqtt` client and exposes helpers like `send_action()` and `set_gear()`. All topics are rooted at the prefix `env0/` — a deliberate single-environment design that keeps the code simple while the system is still being developed.

5.2 PPO AGENT CONFIGURATION

The PPO agent is assembled in `train.py`. The actor and critic networks are defined in `model.py` — keeping them apart lets the value function and the policy learn different representations of the same input, which often helps in practice:

Listing 5.3: Policy network architecture from `model.py`.

```
1 def get_policy_kwargs(ray_count: int = DEFAULT_RAY_COUNT) -> dict:  
2     return dict(  
3         net_arch=dict(  
4             pi=[256, 128],      # actor: obs -> 256 -> 128 -> 3 actions  
5             vf=[256, 128],      # critic: obs -> 256 -> 128 -> scalar value  
6         )  
7     )
```

Back in `train.py`, the model is assembled and a checkpoint callback saves progress every 10% of the training budget. A resume path can be supplied to pick up where a previous run left off:

Listing 5.4: PPO model creation and checkpoint setup from `train.py`.

```
1 if args.resume_model:  
2     model = PPO.load(args.resume_model, env=vec_env,  
3                     learning_rate=args.learning_rate,  
4                     n_steps=args.n_steps, batch_size=args.batch_size)  
5     model.tensorboard_log = args.log_dir  
6 else:  
7     model = PPO("MlpPolicy", vec_env,  
8                 learning_rate=args.learning_rate,
```

```

9         n_steps=args.n_steps, batch_size=args.batch_size,
10        policy_kwargs=get_policy_kwargs(ray_count),
11        verbose=1, tensorboard_log=args.log_dir)
12
13 checkpoint_cb = CheckpointCallback(
14     save_freq=max(total_timesteps // 10, 1000),
15     save_path=args.save_path, name_prefix="shilate_ppo",
16 )

```

Default hyperparameters are listed in Table 5.1.

Table 5.1: Default PPO hyperparameters.

Parameter	Default	CLI flag
Total timesteps	100,000	--total-timesteps
Learning rate	3×10^{-4}	--learning-rate
Rollout steps n	2048	--n-steps
Batch size	64	--batch-size
Clip ε	0.2	SB3 default
Discount γ	0.99	SB3 default
GAE λ	0.95	SB3 default

5.3 TRAINING LOOP AND HEALTH TELEMETRY

One of the more unusual aspects of SHILATE is the structured telemetry that the Python process streams back to the Unity Editor. Instead of just printing progress to a terminal, every meaningful event emits a tagged line that the Editor's parser picks up and displays in the Training Controller window in real time.

The HealthCallback class runs at the end of each rollout. It re-emits Stable-Baselines3 internal metrics as SHILATE-METRIC lines, and checks loss values for NaN or infinity — a surprisingly useful guard during early experimentation with learning rates:

Listing 5.5: Health callback emitting metrics and checking for non-finite losses.

```

1 def _dump_metrics(self) -> None:
2     stats = getattr(self.model.logger, "name_to_value", {}) or {}
3     for key in self._WATCH:
4         if key in stats:
5             val = stats[key]
6             short = key.split("/", 1)[-1]
7             emit("SHILATE-METRIC", f"{short}={val}")
8             if "loss" in key or "kl" in key:
9                 if math.isnan(float(val)) or math.isinf(float(val)):
10                    emit("SHILATE-HEALTH", f"nan_loss key={key} value={
                                val}")

```

The SHILATE-HEALTH lines feed into TrainingHealthEvaluator.cs in the Editor, which updates the health banner colour accordingly. Alongside periodic checkpoints,

the final model is saved on clean exit — or on `KeyboardInterrupt`, because most training runs are stopped manually rather than running to the full timestep limit.

5.4 INFERENCE MODE

Running a trained policy is handled by `ai_driver.py`, which loads a `.zip` checkpoint and feeds live observations into it. The key difference from training is `deterministic=True`: this disables the Gaussian exploration noise added during training, so the policy always picks the action with the highest predicted return. In plain terms — the agent stops experimenting and drives with what it has learned.

The inference loop itself is intentionally simple. It mirrors the same `obs_event.wait()` pattern used in training, but without the Gymnasium wrapper overhead, and it handles episode resets automatically:

Listing 5.6: Inference loop from `ai_driver.py`.

```
1 while _running:
2     obs = build_observation(ctrl, ray_count)
3
4     action, _ = model.predict(obs, deterministic=True)
5     steer, throttle, brake = float(action[0]), float(action[1]), float(
6         action[2])
7     ctrl.send_action(steer, throttle, brake)
8     step += 1
9
10    ctrl.obs_event.clear()
11    if not ctrl.obs_event.wait(timeout=2.0):
12        log.warning("Observation timeout at step %d", step)
13        continue
14
15    if step % 50 == 0:
16        log.info("Step %d | SPD: %.1f km/h | STR: %+.2f | THR: %.2f |
17            BRK: %.2f",
18                step, ctrl.get_speed(), steer, throttle, brake)
19
20    if ctrl.training_done:
21        episode += 1
22        log.info("Episode %d complete (step %d)", episode, step)
23        step = 0
24        time.sleep(0.5)
25        ctrl.send_reset()
26        ctrl.obs_event.clear()
27        ctrl.obs_event.wait(timeout=5.0)
```

Running the inference loop on the deployed Leda image requires no changes to the code — only the `--model` path argument changes. This is the same container image used during development, rebuilt and pushed to the device registry by `provision-device.sh`.

5.5 CROSS-PLATFORM PROCESS MANAGEMENT

Because Unity typically runs on Windows while the Python process runs inside WSL2, `PythonProcessManager.cs` handles virtual-environment activation differently per platform. On Windows it calls `Scripts\activate`; on Linux/macOS it uses `bin/activate`. Graceful shutdown sends `SIGTERM` to the process group so TensorBoard logs are flushed and the final checkpoint is written before Play mode exits. A hard kill is used as a fallback if the process does not respond in time.

6. SDV INTEGRATION WITH ECLIPSE LEDA

6.1 ECLIPSE LEDA OVERVIEW

Eclipse Leda [1] is an SDV distribution built on the Yocto Project. It packages a curated set of Eclipse SDV middleware components — Mosquitto MQTT broker, Kuksa databroker, Kanto container manager, and OpenTelemetry collector — into a bootable image that runs on QEMU (x86-64) or on supported ARM hardware such as the Raspberry Pi 4. Its primary purpose is to serve as a reference platform for developing and validating SDV software stacks before deploying to a real vehicle ECU.

SHILATE targets the QEMU image (`sdv-image-all-qemux86-64.wic.qcow2`) included in the repository. The `leda/run-leda.sh` script launches the image with a bridged network interface so that the host machine and the QEMU guest share a Mosquitto broker at `127.0.0.1:1883`.

6.2 MQTT-TO-KUKSA BRIDGE

The `mqtt-kuksa-feeder` container subscribes to the `vehicle/#` wildcard on the Mosquitto broker and forwards each received message to the Kuksa databroker [9] as a VSS path write via gRPC. The topic-to-VSS mapping is shown in Table 6.1.

Table 6.1: MQTT topic to VSS path mapping.

MQTT topic	VSS path
<code>vehicle/speed</code>	<code>Vehicle.Speed</code>
<code>vehicle/rpm</code>	<code>Vehicle.Powertrain.CombustionEngine.Speed</code>
<code>vehicle/steering</code>	<code>Vehicle.Chassis.SteeringWheel.Angle</code>
<code>vehicle/brake</code>	<code>Vehicle.Chassis.Brake.PedalPosition</code>
<code>vehicle/throttle</code>	<code>Vehicle.OBD.ThrottlePosition</code>

After the feeder writes a signal, any subscriber — including the Velocitas vehicle application — is notified with the new value within the same event loop tick.

The core of the bridge is the `_on_message` callback. Each time a Unity MQTT publish arrives, the callback parses the JSON payload, resolves the VSS path from the pre-built lookup table, and writes the value to Kuksa via gRPC in one call:

Listing 6.1: MQTT-to-Kuksa bridge callback from `feeder.py`.

```

1 def _on_message(self, client, userdata, msg: mqtt.MQTTMessage):
2     vss_path = self._topic_to_vss.get(msg.topic)
3     if vss_path is None:
4         return # unknown sub-topic, ignore
5 
```

```
6     try:
7         payload = json.loads(msg.payload)
8         value = float(payload["value"])
9     except (json.JSONDecodeError, KeyError, ValueError) as exc:
10        log.debug("Bad payload on %s: %s (%s)", msg.topic, msg.payload,
11                exc)
12        return
13
14    try:
15        self._kuksa.set_value(vss_path, value)
16    except Exception as exc:
17        log.warning("KUKSA write failed for %s: %s", vss_path, exc)
18        self._kuksa.disconnect()
19        self._kuksa.connect()    # attempt to reconnect on next call
```

The reconnect-on-failure logic inside `set_value` means the feeder survives a brief Kuksa databroker restart without manual intervention — important when the container orchestrator restarts services after a firmware update.

6.3 VELOCITAS VEHICLE APPLICATION

The Velocitas vehicle application [10] implements the operational safety monitor for the RL agent. It subscribes to five VSS signals via the Kuksa databroker, logs a telemetry summary every 5 seconds, and enforces two hard limits:

- **Overspeed:** if `Vehicle.Speed` exceeds 120 km/h, an alert is published to `leda/command/alert` with payload `{"type": "overspeed", "value": <speed>}`.
- **RPM redline:** if engine RPM exceeds 6500, an analogous alert is published.

Alerts are currently informational; the architecture is designed so that the safety monitor can be upgraded to a command-injection path without modifying any other component.

The subscription setup and the safety check callbacks for overspeed are shown below. The pattern is identical for the RPM redline check:

Listing 6.2: Velocitas subscription setup and overspeed handler from `app.py`.

```
1 async def on_start(self):
2     await vehicle.Speed.subscribe(self._on_speed)
3     await vehicle.Powertrain.CombustionEngine.Speed.subscribe(self.
4         _on_rpm)
5     await vehicle.Chassis.SteeringWheel.Angle.subscribe(self.
6         _on_steering)
7     await vehicle.Chassis.Brake.PedalPosition.subscribe(self._on_brake)
8     await vehicle.OBD.ThrottlePosition.subscribe(self._on_throttle)
9     asyncio.ensure_future(self._telemetry_summary_loop())
10
11 async def _check_overspeed(self, speed: float):
12     if speed > SPEED_LIMIT and not self._overspeed_active:
13         self._overspeed_active = True
```



```
12         alert = {
13             "type":      "overspeed",
14             "speed":     round(speed, 2),
15             "limit":     SPEED_LIMIT,
16             "message":   f"Speed {speed:.1f} km/h exceeds limit {
17                             SPEED_LIMIT:.0f} km/h",
18         }
19         log.warning("ALERT: %s", alert["message"])
20         await self.publish_event(ALERT_TOPIC, json.dumps(alert))
21     elif speed <= SPEED_LIMIT and self._overspeed_active:
22         self._overspeed_active = False
23         log.info("Speed returned to normal: %.1f km/h", speed)
```

Two details are worth noting. First, the flag `_overspeed_active` prevents a flood of repeated alert messages when the vehicle is continuously above the threshold — only the first crossing and the recovery generate a message. Second, the alert payload is sent to `leda/command/alert`, a topic that Unity already subscribes to via the `leda/command/#` wildcard in `LedaBroker.cs`, so the Editor health window will surface it without any additional wiring.

6.4 CONTAINER DEPLOYMENT

All three service containers (`leda-controller`, `mqtt-kuksa-feeder`, `velocitas-app`) ship with Dockerfiles producing minimal Python 3.11 slim images. Each is described by a Kanto container manifest (`kanto-manifest.json`) specifying the OCI image reference, environment variables (`MQTT_HOST`, `MQTT_PORT`, `MODE`, `MODEL_PATH`), and restart policy. Provisioning is automated by `leda/provision-device.sh`, which uses SSH to copy manifests to the Leda device and invoke the Kanto CLI to pull and start the containers.

6.5 LEDA TELEMETRY BROKER

In addition to MQTT, the Leda image runs a lightweight telemetry broker implemented in Rust. The broker listens on TCP port 7878 and acts as a multiplexing relay: any client that connects and sends telemetry data has that data forwarded to all other connected clients. This enables tools such as `netcat` or custom dashboards to observe the live telemetry stream without subscribing to individual MQTT topics.

6.6 RASPBERRY PI 4 HARDWARE ACTUATOR

Closing the loop between the SDV software layer and a real physical device requires hardware that can translate a VSS signal into a mechanical action. The design described in this section is theoretical — the physical build has not been assembled — but it is grounded in components that are readily available and well understood.

The target application is a 12 V DC water pump driven by a pulse-width-modulated (PWM) signal, where the duty cycle is set by the RL agent's throttle output via the VSS path `Vehicle.OBD.ThrottlePosition`.

6.6.1 CIRCUIT DESIGN

A Raspberry Pi 4B [15] runs Eclipse Leda on its ARM processor and therefore participates in the same SDV software stack as the QEMU simulation. Its 3.3 V GPIO pins cannot drive a 12 V motor load directly, so a MOSFET module is interposed. A suitable choice is the IRF520N N-channel MOSFET: its gate threshold voltage is well within the 3.3 V logic range, its $R_{DS(on)}$ is low enough (approx. $0.27\ \Omega$) to avoid significant dissipation at pump currents, and its V_{DS} rating of 100 V comfortably exceeds the 12 V supply.

The gate is driven through a $1\ \text{k}\Omega$ series resistor to limit inrush current and damp potential ringing on the gate trace. A small pull-down resistor ($10\ \text{k}\Omega$) from gate to ground ensures the MOSFET is fully off when the GPIO pin is tri-stated at boot, preventing unintended motor activation during start-up.

The pump motor is an inductive load. When the MOSFET switches off abruptly, the energy stored in the motor's inductance L has nowhere to go and appears as a voltage spike across the switching node:

$$V_{\text{spike}} = -L \frac{dI}{dt}. \quad (6.1)$$

Without protection, this spike can exceed the MOSFET's V_{DS} rating and destroy the transistor. A *flyback suppressor* — also called a freewheeling or snubber diode — provides a low-impedance current path that clamps the spike. A 1N4007 general-purpose rectifier connected in reverse across the pump terminals (cathode to the 12 V rail, anode to the drain) is sufficient for modest pump currents. For faster PWM frequencies or higher currents, a Zener diode or a dedicated TVS (Transient Voltage Suppressor) device placed in the same position offers a tighter voltage clamp.

The complete circuit, described schematically, is:

RPI GPIO23 $\xrightarrow{1\ \text{k}\Omega}$ **MOSFET Gate** — **MOSFET Drain** → **Pump (-); Pump (+)** → **12 V**; **1N4007/TVS** from Drain to 12 V (reverse-biased); **MOSFET Source** → **GND**; **10 k Ω pull-down** from Gate to GND.

6.6.2 SOFTWARE: VSS SUBSCRIPTION TO PWM OUTPUT

The Raspberry Pi runs a small Python daemon alongside the Leda containers. It subscribes to the Kuksa databroker using the `kuksa-client` library and, on each value change, converts the normalised throttle reading to a PWM duty cycle for GPIO 23:

Listing 6.3: Theoretical Raspberry Pi 4 pump controller subscribed to VSS via Kuksa.

```
1 import asyncio
2 import RPi.GPIO as GPIO
3 from kuksa_client.grpc.aio import VSSClient
4
5 PUMP_GPIO = 23
6 PWM_FREQ = 1000    # Hz -- 1 kHz carrier well above audible range
7 VSS_PATH = "Vehicle.OBD.ThrottlePosition"
```

```
8 KUKSA_ADDR = ("127.0.0.1", 55555)
9
10 GPIO.setmode(GPIO.BCM)
11 GPIO.setup(PUMP_GPIO, GPIO.OUT)
12 pwm = GPIO.PWM(PUMP_GPIO, PWM_FREQ)
13 pwm.start(0)
14
15 async def main() -> None:
16     async with VSSClient(*KUKSA_ADDR) as client:
17         async for updates in client.subscribe_current_values([VSS_PATH]):
18             :
19             for path, value in updates.items():
20                 # ThrottlePosition is 0.0-100.0 (percent); map to duty
21                 cycle
22                 duty = max(0.0, min(100.0, float(value.value)))
23                 pwm.ChangeDutyCycle(duty)
24
25 asyncio.run(main())
```

The Kuksa gRPC subscription fires immediately on value change, so the pump duty cycle tracks the agent's throttle output with sub-millisecond software latency. The PWM carrier frequency of 1 kHz is above the audible range of most motors, reducing acoustic noise without requiring an especially low MOSFET gate capacitance.

6.6.3 CONNECTION TO SHILATE

Within the SHILATE data flow, the RL agent running on the host machine publishes a throttle action to the MQTT topic `env0/vehicle/control/throttle`. The `mqtt-kuksa-feeder` container forwards this to the Kuksa databroker as `Vehicle.OBD.ThrottlePosition`. The RPi daemon described above subscribes to that path and drives the pump accordingly.

This is precisely the Software-Defined Vehicle pattern: the physical actuator has no knowledge of the RL algorithm, the simulation, or the MQTT broker. It speaks only VSS signals. Replacing the pump with any other VSS-aware actuator — a servo, a heater, a relay — requires no changes to the Python training code.

7. EXPERIMENTS AND RESULTS

7.1 EXPERIMENTAL SETUP

All training experiments were conducted on a workstation running Windows 11 with the Unity Editor invoked via WSL2 for the Python process. The environment is summarised in Table 7.1.

Table 7.1: Experimental environment.

Component	Specification
CPU	Intel Core i7-12700H (14 cores)
RAM	32 GB DDR5
GPU	NVIDIA RTX 3070 Ti (8 GB)
OS	Windows 11 / WSL2 Ubuntu 22.04
Unity	2022.3 LTS
Python	3.10.12
Stable-Baselines3	2.3.0
PyTorch	2.1.0
Gymnasium	0.29.1
MQTT broker	Mosquitto 2.0.18

The Mosquitto broker ran on the host at 127.0.0.1:1883. Unity physics ran at the default fixed time-step of 0.02 s (50 Hz) with `Time.timeScale = 1` (real time). The Python process executed in a WSL2 terminal and communicated with Unity exclusively over the loopback interface.

7.2 TRAINING CONVERGENCE

A baseline training run of 100,000 timesteps was conducted using the default hyperparameters from Table 5.1. The episode reward mean, computed over a rolling 10-episode window as reported by Stable-Baselines3, is the primary convergence indicator.

At the start of training the agent explores nearly uniformly, producing episode rewards in the range $[-5, +10]$. By approximately 20,000 timesteps the agent consistently avoids the inner wall and begins accumulating progress rewards, with the rolling mean rising to $[20, 40]$. The half-turn bonus of $+25$ is reliably triggered from around 30,000 timesteps. By 80,000 timesteps the agent completes full laps in the majority of episodes and the rolling mean stabilises in the $[60, 80]$ range. Table 7.2 summarises milestones across three independent seeds.

Table 7.2: Training convergence milestones (mean over three seeds).

Milestone	Timestep	Std. dev.
First collision-free episode	12,400	± 2100
Half-turn bonus reliably triggered	31,000	± 3800
First full lap completed	48,500	± 5200
Rolling mean ≥ 60	76,000	± 6900

7.3 REWARD SHAPING ABLATION

To evaluate the contribution of the half-turn milestone bonus, a second training run was performed with $c_h = 0$ (all other parameters unchanged). Without the half-turn bonus, the first complete lap was delayed by approximately 18,000 timesteps on average and the rolling mean reward at 100,000 steps was approximately 12 points lower. This confirms that the intermediate milestone provides a useful progress signal early in training.

A further ablation with reduced collision penalty ($c_c = -1$ instead of -5) produced an agent that completed laps but drove closer to the inner wall, indicating that the stronger penalty is necessary to encourage a safety margin.

7.4 INFERENCE EVALUATION

The `best_model.zip` checkpoint was evaluated over 50 episodes using `ai_driver.py`. Results are summarised in Table 7.3.

Table 7.3: Inference evaluation over 50 episodes.

Metric	Value
Lap completion rate	82% (41/50)
Mean episode reward	63.4
Mean lap duration	38.2 s
Mean speed (completed laps)	47.6 km/h
Collisions per episode	0.6
Timeout episodes	9 (18%)

The 18% of episodes that ended in timeout typically corresponded to the agent entering a slow-speed oscillation near an obstacle, suggesting that the current reward shaping does not sufficiently penalise stalling. This is identified as a direction for future improvement.

7.5 HEALTH MONITORING VALIDATION

The health monitoring system was validated by inducing controlled failures:

- **Observation timeout.** The Unity simulation was paused mid-episode. The health evaluator correctly transitioned to *Critical* after 5 seconds with the message “No observation received for 5 s”.

- **Python crash.** The Python process was killed with SIGKILL. The Editor detected the non-zero exit code and displayed “Python process exited unexpectedly (exit code 137)”.
- **MQTT disconnect.** Mosquitto was stopped. The EditorMqttListener raised a connection-lost event within approximately 35 seconds (one keep-alive period) and the banner transitioned to *Disconnected*.

All three failure modes were correctly detected and displayed without manual intervention.

7.6 END-TO-END SDV INTEGRATION TEST

The full deployment chain was validated on the QEMU-hosted Leda image. With all three containers running, the following checks were performed:

1. A `Kuksa GetValue` call for `Vehicle.Speed` returned the current simulator speed within 100 ms of it being published to MQTT, confirming correct feeder operation.
2. When the Python agent applied maximum throttle for 5 seconds, the `Velocitas` application published an overspeed alert to `leda/command/alert`, confirming end-to-end signal propagation from the simulation through the VSS layer to the safety monitor.
3. The alert was received by a test `mosquitto_sub` subscriber on the host machine, confirming that the outbound MQTT path from the Leda image to the host network was functional.

8. CONCLUSIONS

8.1 SUMMARY OF CONTRIBUTIONS

This thesis presented SHILATE, an open-source pipeline that trains a deep reinforcement learning agent inside a Unity physics simulation and deploys the resulting policy onto an Eclipse Leda SDV runtime stack. The main contributions are as follows.

An integrated simulation environment for SDV-targeted RL. A Unity 2022 LTS scene was constructed programmatically, comprising a realistic rear-wheel-drive vehicle physics model, a 21-ray LiDAR-style sensor, and a circular obstacle course. The environment publishes observations and consumes control commands through MQTT at 10 Hz, using the same protocol and topic namespace as the deployment target, so that no protocol translation is required when moving from training to deployment.

A shaped reward function with quantified milestone bonuses. A four-term reward signal was designed and ablated: a continuous progress reward, a collision penalty, a half-turn milestone bonus, and a lap-completion bonus. Ablation experiments confirmed that the intermediate milestone bonus accelerates convergence by approximately 18,000 timesteps and that the magnitude of the collision penalty directly controls the agent's safety margin.

A Gymnasium/SB3 training pipeline with live Editor integration. The MQTT interface is wrapped in a Gymnasium-compliant environment, allowing any Stable-Baselines3 algorithm to be substituted without modification. A Unity Editor window provides one-click training control, real-time health monitoring across five independently evaluated failure modes, and live metric graphs.

A complete SDV deployment chain. A containerised MQTT-to-Kuksa bridge maps simulator telemetry to COVESA VSS paths, and a Velocitas vehicle application enforces operational limits on the running RL policy. The entire chain was validated end-to-end on a QEMU-hosted Eclipse Leda image.

8.2 LIMITATIONS

Single training environment. SHILATE supports exactly one Unity Editor instance at real time (`Time.timeScale = 1`). Training throughput is therefore limited by wall-clock simulation speed.

No sim-to-real gap mitigation. The sensor model, vehicle physics, and obstacle geometry are all abstractions. No domain randomisation or adaptation mechanism is included, so a policy trained in SHILATE is unlikely to transfer directly to a physical vehicle without further fine-tuning.

Informational-only safety monitor. The Velocitas application publishes alerts but does not currently override the agent's control commands. The deployment chain provides monitoring rather than enforcement.

Limited action space. The current action space does not include gear selection or parking-brake inputs, restricting the range of manoeuvres available to the agent.

8.3 FUTURE WORK

Multi-environment parallelisation. Replacing the single Unity Editor instance with headless builds or container-based simulation instances would dramatically increase sample efficiency and enable distributed RL algorithms such as IMPALA.

Curriculum learning. Automatically varying the obstacle configuration as the agent improves would implement a difficulty curriculum and is expected to address the stalling behaviour observed in the inference evaluation.

Physical hardware deployment. Validating the deployment chain on a physical SDV platform — such as a CAN-bus-instrumented RC vehicle running Eclipse Leda on embedded hardware — would provide direct evidence of the pipeline's applicability to real automotive systems.

Safety monitor upgrade. Extending the Velocitas application to publish override control commands when a limit is exceeded would close the loop from detection to intervention.

Richer observation space. Adding front-facing camera images or occupancy-grid representations would bring the perception model closer to production ADAS systems, at the cost of increased training complexity.

9. BIBLIOGRAPHY

- [1] Eclipse Leda Project, *Eclipse leda: SDV distribution documentation*, Accessed June 2026, 2023. [Online]. Available: <https://eclipse-leda.github.io/leda/>.
- [2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017. [Online]. Available: <https://arxiv.org/abs/1707.06347>.
- [3] B. R. Kiran, I. Sobh, V. Talpaert, *et al.*, "Deep reinforcement learning for autonomous driving: A survey," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 6, pp. 4909–4926, 2021. DOI: 10.1109/TITS.2021.3054625.
- [4] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018. [Online]. Available: <http://incompleteideas.net/book/the-book-2nd.html>.
- [5] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021. [Online]. Available: <http://jmlr.org/papers/v22/20-1364.html>.
- [6] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in *Proceedings of the 1st Annual Conference on Robot Learning (CoRL)*, 2017, pp. 1–16. [Online]. Available: <https://arxiv.org/abs/1711.03938>.
- [7] A. Juliani, V.-P. Berges, E. Teng, *et al.*, "Unity: A general platform for intelligent agents," *arXiv preprint arXiv:1809.02627*, 2020. [Online]. Available: <https://arxiv.org/abs/1809.02627>.
- [8] COVESA – Connected Vehicle Systems Alliance, *Vehicle signal specification (VSS)*, Accessed June 2026, 2023. [Online]. Available: https://covesa.github.io/vehicle_signal_specification/.
- [9] Eclipse Kuksa Project, *Eclipse Kuksa.VAL documentation*, Accessed June 2026, 2024. [Online]. Available: <https://eclipse-kuksa.github.io/kuksa.val/>.
- [10] Eclipse Velocitas Project, *Eclipse velocitas documentation*, Accessed June 2026, 2024. [Online]. Available: <https://eclipse-velocitas.github.io/velocitas-docs/>.
- [11] OASIS, "MQTT version 3.1.1," OASIS Standard, Tech. Rep., 2014. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html>.
- [12] G. Brockman, V. Cheung, L. Pettersson, *et al.*, "OpenAI gym," *arXiv preprint arXiv:1606.01540*, 2016. [Online]. Available: <https://arxiv.org/abs/1606.01540>.

- [13] M. Towers, A. Kwiatkowski, J. K. Terry, *et al.*, “Gymnasium: A standard interface for reinforcement learning environments,” *arXiv preprint arXiv:2407.17032*, 2024. [Online]. Available: <https://arxiv.org/abs/2407.17032>.
- [14] Unity Technologies, *Unity manual — unity 2022 LTS*, Accessed June 2026, 2022. [Online]. Available: <https://docs.unity3d.com/2022.3/Documentation/Manual/>.
- [15] Raspberry Pi Ltd, *Raspberry Pi 4 Model B*,
[urlhttps://www.raspberrypi.com/products/raspberry-pi-4-model-b/](https://www.raspberrypi.com/products/raspberry-pi-4-model-b/), Accessed: 2025, 2023.